PROGRAMMING & CS FUNDAMENTALS

Sorting Algorithms Visual Guide

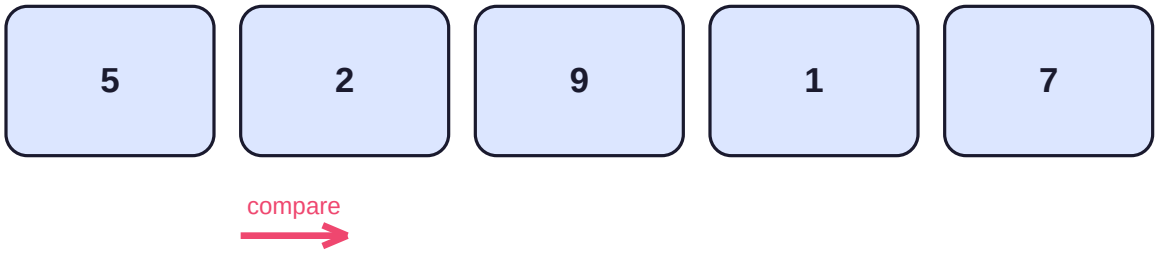
From bubble to quicksort —
what O(n log n) really costs and buys.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/7

The Comparison Model

Sorting by comparing pairs — the arena every classic sort fights in.



Decision tree: n! possible orders -> any comparison sort needs >= log2(n!) ~ n log n comparisons.

```
n=8:   8! = 40320 orders -> need >= 16 comparisons
n=1M:  n log n ~ 20M comparisons minimum
escape hatch: non-comparison sorts (counting/radix) beat it with extra assumptions
```

The n log n lower bound applies ONLY to comparison sorts. Counting sort hits O(n+k) by never comparing — keys must be small integers.

STEP 2/7

The O(n^2) Trio

Simple, slow, and still useful for tiny n.

```
bubble:    swap adjacent inversions, repeat     # early-exit if no swaps
selection: pick min of rest, swap into place   # fewest writes: O(n)
insertion: grow sorted prefix, insert next     # O(n) on nearly-sorted!
```

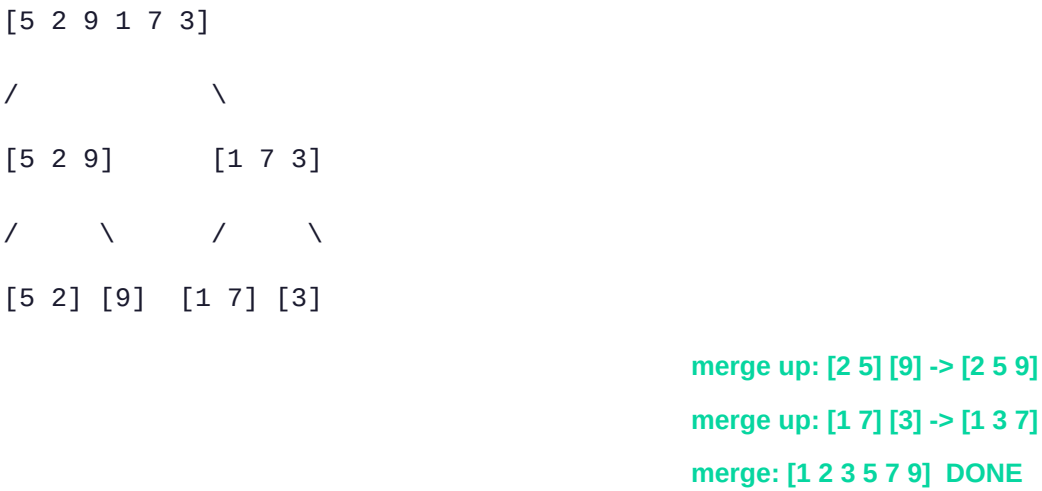
insertion sort on [1,2,3,5,4]: one shift — nearly O(n). On reversed: O(n^2).

- Adaptive: insertion sort exploits existing order — why Timsort uses it for small runs.
- Selection sort: exactly n swaps — pick it when writes are expensive (flash memory).
- All three are in-place O(1) space and stable-ish; all die past n ~ 10^4.

STEP 3/7

Merge Sort: Divide & Conquer

Split, sort, merge — guaranteed O(n log n), stable.




```
T(n) = 2T(n/2) + O(n)   # two halves + linear merge
=> O(n log n) always, O(n) extra space for merging
```

- Stable: equal elements keep their order — merge takes from the left first.
- External sorting: merge sort streams from disk — the database sort.
- Predictable: no bad inputs, unlike quicksort — pick it when worst case matters.

STEP 4/7

Quicksort: Partition & Conquer

Pick a pivot, partition around it, recurse — the practical champion.

```
[5 2 9 1 7 3]   pivot = 5

partition ->  [2 1 3]  5  [9 7]     # < pivot | pivot | > pivot

recurse   ->  [1 2 3]  5  [7 9]
```

```
Lomuto: pivot = last; i tracks boundary of '< pivot'
Hoare:  two fingers walk inward, swap inversions  # fewer swaps
avg O(n log n); worst O(n^2) on sorted input + bad pivot
```

- Pivot choice is everything: random or median-of-3 defeats adversarial input.
- In-place, cache-friendly — why it's the default in most standard libraries.
- Introsort (std::sort): quicksort + heapsort fallback when recursion goes too deep.
- 3-way partition (Dutch flag): O(n) on arrays with many duplicates.

STEP 5/7

Heap Sort: The Heap Does the Work

Heapify, then repeatedly extract-max to the end.

```
heapify(arr)                # O(n): max-heap in place
for end in reversed(range(n)):
    swap(arr[0], arr[end])  # max -> its final slot
    sift_down(0, end)       # restore heap on prefix
# O(n log n) worst case, O(1) space — but NOT stable
```

- Guaranteed O(n log n) with O(1) space — the safe in-place choice.
- Not stable: equal elements can leapfrog during swaps.
- Poor cache behavior vs quicksort: sifting jumps across the array — slower in practice.
- Interview use: 'sort with O(1) space and guaranteed n log n' -> heapsort.

STEP 6/7

Stability & Choosing

Equal keys: does their original order survive?

Quicksort — avg n log n

unstable* · fastest in practice

Merge sort — n log n

stable · needs O(n) space

Heap sort — n log n

unstable · O(1) space guarantee

Timsort — n log n

stable · Python/Java default

Counting/radix — n + k

stable · non-comparison, bounded keys

*quicksort can be stabilized with index tie-breaks, at a cost.

STEP 7/7

Sorting Wisdom

Rules that survive contact with production.

- n under ~50: insertion sort wins — constants beat asymptotics (Timsort knows this).
- Need stability (sort by name, then by age)? Stable sort or composite keys.
- Nearly sorted input: insertion O(n); quicksort with naive pivot degrades to O(n^2).
- Huge data, tiny RAM: external merge sort — stream runs from disk.
- Just call the stdlib sort: decades of tuning (introsort/Timsort) beat hand-rolled code.
- Sorting is the gateway: binary search, two pointers, and dedup all assume sorted input.

STEP ✓I✓

Key Takeaways

Sorting Algorithms — Remember

- Comparison sorts can't beat n log n (decision-tree bound); counting/radix escape via assumptions.
- Merge sort: guaranteed O(n log n), stable, O(n) space — the safe divide-and-conquer.
- Quicksort: average O(n log n), in-place, cache-hot — but pivot choice decides worst case.
- Heap sort: O(n log n) worst case in O(1) space, unstable, cache-cold.
- Stability matters when equal keys carry order; defaults: Timsort (Python/Java), introsort (C++).

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev